

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Trabajo Integrador - Programación 1

Alumnos

Laura Analía Portela C-18, Daniel Elías Buscaglia C-12

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Repositorio en GitHub: [https://github.com/Laport71/TP_Integrador_Programacion1_Paises]

Video Demostrativo: [<https://www.youtube.com/watch?v=RPVDURkQGKg&t=1s>]

Índice

Índice.....	2
Objetivo.....	3
Marco Teórico	3
Desarrollo.....	5
Manejo de información.....	5
Proceso de validación de datos ingresados por el usuario	6
Funciones en el código.....	7
Proceso de búsqueda y validación de similitud	8
Orden de diccionarios dentro de listas	8
Estadísticas de población mínima y máxima.....	8
Manejo de Archivo.csv	8
Decisiones Técnicas y Arquitectura	9
Esquema del menú.....	9
Organigrama de cada opción del menú	10
Capturas de pantalla	11
Dificultades	12
Conclusiones	13
Referencias.....	14

Objetivo

Desarrollar una aplicación en Python que permita gestionar información sobre países. El sistema debe ser capaz de leer datos desde un archivo CSV, realizar consultas y generar indicadores clave a partir del dataset.

Marco Teórico

Para el desarrollo de este proyecto se utilizaron distintos conceptos y herramientas del lenguaje de programación Python. A continuación se explica brevemente cada uno de ellos.

1)Listas, tuplas, Diccionarios En este proyecto se usó como estructura principal una lista de diccionarios para almacenar la información de cada país, donde las claves eran nombre, población, superficie y continente. A continuación se listan las variables más importantes utilizadas.

base_paises

Es la estructura más importante del programa. Comienza como una lista vacía y se va completando con diccionarios a medida que el usuario carga países. Cada diccionario dentro de ella representa un país y tiene exactamente cuatro claves:

```
{"pais": "Argentina", "poblacion": 46000000, "superficie": 2780400, "continente": "América"}
```

Esta lista es el centro de todas las operaciones del programa: se usa para buscar, filtrar, ordenar, actualizar y calcular estadísticas. También es lo que se guarda y se lee desde el archivo paises.csv cada vez que el programa inicia o finaliza.

lista_paises

Es una tupla de strings que contiene los 195 países soberanos reconocidos internacionalmente, incluyendo los 193 miembros de la ONU más la Santa Sede y el Estado de Palestina. Cuando el usuario ingresa un nombre de país, el programa lo compara contra esta lista para verificar si existe.

lista_continentes

Es una tupla de strings con los 5 continentes del modelo adoptado por la ONU: América, Asia, África, Europa y Oceanía. También cumple función de validación.

dic_paises (diccionario temporal)

Se crea al leer el archivo paises.csv al inicio del programa. Por cada línea del archivo se arma un diccionario con las cuatro claves y se agrega inmediatamente a base_paises. Es temporal: cumple su función y luego es reemplazado por el del siguiente país.

continentes (lista auxiliar)

Se genera dentro de las funciones de estadísticas. El programa recorre base_países y va guardando en esta lista los continentes que encuentra, sin repetir. Se usa para calcular promedios de población y superficie por continente.

suma_poblacion_continente / **suma_superficie_continente** (listas auxiliares correlativas)

Son listas paralelas a continentes: lo que está en la posición 0 de continentes corresponde a lo que está en la posición 0 de estas listas. Almacenan el promedio calculado de población o superficie para cada continente. Se recorren juntas al final para imprimir los resultados.

países (lista auxiliar en cantidad_países_continente)

Lista temporal que se resetea en cada vuelta del bucle. Acumula los nombres de países que pertenecen al continente que se está analizando en ese momento, y al final se usa su longitud con len() para contar cuántos países tiene ese continente.

2) Validación de datos con Try-Except Para evitar que el programa se detenga por un error.

3) Funciones En este proyecto, los menús de opciones fueron definidos dentro de funciones para poder reutilizarlos cuando fuera necesario.

- **Función any() y enumerate()** Se utilizó para verificar rápidamente si un país existía antes de realizar una búsqueda completa. La función enumerate() permite recorrer una lista obteniendo al mismo tiempo el índice y el valor de cada elemento, lo cual fue útil para encontrar la posición exacta de un país y poder actualizar sus datos.

- **Ordenamiento con sorted() y funciones lambda** La función sorted() devuelve una lista ordenada según el criterio que se le indique. Las funciones lambda son funciones pequeñas que se escriben en una sola línea y no tienen nombre, muy útiles para indicarle a sorted() por qué clave ordenar los diccionarios. Por ejemplo, para ordenar países por población de mayor a menor se usó reverse=True.

- **Funciones min() y max()** Estas funciones permiten encontrar el valor mínimo y máximo dentro de una lista. Al trabajar con listas de diccionarios es necesario indicarles qué clave comparar mediante una función lambda.

4) Archivos CSV En este proyecto se usó un archivo llamado países.csv para guardar los datos ingresados, de modo que no se pierdan al cerrar el programa. Cada vez que el usuario agrega o modifica información, el archivo se actualiza automáticamente.

Desarrollo

Manejo de información

Para cumplir el objetivo, el equipo decidió optar por implementar dos tuplas.

- lista_países: contiene los 195 países soberanos reconocidos internacionalmente
- lista_continentes: contiene el modelo de 5 continentes

Estas sirvieron de base para comparar el ingreso del usuario, ya que puede introducir cualquier tipo de dato y lo que se esperan son países y continentes, en caso de ingresar un string que no está en alguna de las listas se le indica al operador que puede optar por corregir el ingreso o continuar sin modificarlo

```
4.- Verificar países
5.- Ordenar países
6.- Mostrar estadísticas.
7.- Salir

- Actualización del registro de países -

a continuación se le solicitara que ingrese la cantidad de países
que desea guardar en la base de datos, seguido de la información
en el orden establecido:
- Nombre.
- Población.
- Superficie en Km².
- Continente.

Cuántos países va a agregar a la lista?
ingrese la cantidad con números: 2
ingrese el nombre del país: Carlos
IMPORTANTE: estas agregando a la lista un país que no existe,
asegurate de ingresarlo de forma correcta, revisa los acentos y espacios.

quieres agregarlo de todos modos? s/n: █
```

Luego se estableció una lista de diccionarios que almacena toda la información que ingresa el usuario en el siguiente orden:

- nombre,poblacion,superficie,continente

El cual es fundamental respetarlo ya que se utilizará en el dataset del archivo.csv

Una vez establecida la forma en la que ingresarían los datos al sistema se definieron los distintos menús de opciones, estos se alojaron dentro de funciones, de esta forma podrían ser utilizados llamando a la función ahorrando así líneas de código, dado que podría ser necesario repetirlos varias veces en el proceso.

Luego de establecer las distintas opciones a las que podría acceder el usuario, se procedió a generar el código necesario para procesar los datos y obtener la información.

Proceso de validación de datos ingresados por el usuario

El primer paso por el que debe pasar el usuario es el de ingresar países al sistema, opción 1 del menú principal. Para esto se implementó una captura de errores (bloque **try - except**) dentro de un bucle (**while**) esto permite validar los ingresos del usuario, ya que es posible generar un error (palabra clave **raise** seguido del error que se desea generar) para limitar el ingreso del usuario.

Por ejemplo:

- al momento de solicitar un int para la cantidad de países que ingresaran a la lista el usuario podría utilizar un numero negativo o 0, el programa no arrojará ningún error ni se detendrá; simplemente saltará el bloque de código y continuará con las siguientes instrucciones.

Para forzarlo a recorrer el bucle for creamos un error utilizando la palabra clave raise

```
182     while True:
183         # iniciamos la captura de posibles errores:
184         try:
185             # se solicita una cantidad para saber cuantos paises
186             # va a guardar el usuario:
187             print("Cuantos paises va a agregar a la lista?")
188             cantidad = int(input("ingrese la cantidad con numeros: "))
189             # en caso de que ingrese 0 o un numero negativo:
190             if cantidad <= 0:
191                 # se genera un error por mas que el ingreso sea un valor entero (int):
192                 raise ValueError
193             # en caso de error por valor:
194             except ValueError:
195                 print("\nError: debe ingresar un numero entero mayor a 0\n")
196             # si el ingreso es valido:
197             else:
198                 # se rompe el bucle
199                 break
```

Luego de validar cada ingreso aplicando los mismos pasos, todos los datos se procesan y son alojados en una lista de diccionarios llamada base_paises.

Una vez realizada la carga se le permite al usuario ingresar a cualquier opción del menú, permitiendo actualizar, buscar, filtrar, ordenar o mostrar estadísticas.

Funciones en el código

Las opciones y los menús de cada instancia fueron definidos dentro de funciones, las cuales a excepción de **agregar_pais()** están condicionadas por un bloque de control de flujo (**if-elif-else**) el cual no le permite al usuario hacer uso completo de las mismas hasta que la lista de diccionarios tenga contenido, en caso de que sea la primera vez que se ejecuta el programa o por error haber borrado el archivo `países.csv`

```
# _____#
# - opcion 6 del menu de principal -
# - opcion 3 del menu de organizar países -
# la siguiente funcion obtiene el promedio de superficie por continente:

def promedio_superficie_continente():
    # si se existe algun pais dentro de la lista:
    if base_paises:
        # primero se obtienen todos los continentes dentro de la lista de diccionarios
        # se guardan dentro de la lista continentes:
        continentes = []
        # se crea una lista que es corelativa a la lista continentes:
        suma_superficie_continente = []
```

Cada vez que el usuario realiza la acción de buscar, actualizar, filtrar o ordenar los países, el programa primero realiza una búsqueda rápida utilizando la función `any()` ya que recibe un iterable al que se le indica que busquemos y que parámetros necesita para encontrarlo dentro de una lista, diccionario o tupla, en caso de encontrarlo devuelve un booleano.

En caso de que la funcion `any()` devuelva `True` desempaquetamos todos los elementos de la lista de diccionarios utilizando un bucle `For` esto permite recorrer cada uno de los diccionarios y obtener el valor de las claves en ellos, para ello hacemos uso del método `.get()` que permite indicar la clave a obtener del diccionario, en este caso puede ser “pais”, “poblacion”, “superficie” o “continente” sin que rompa el código en caso de no encontrarse la clave.

```
if any(busqueda.lower() in x["pais"].lower() for x in base_paises):
    # en caso de que devuelva True:
    for x in base_paises:
        # recorreremos los diccionarios en la lista base_paises
        # obtenemos los valores de las claves:
        pais = x.get("pais")
        poblacion = x.get("poblacion")
        superficie = x.get("superficie")
        continente = x.get("continente")
        # si lo que ingreso el usuario coincide con el nombre completo
        # o parte de el:
```

Cuando el usuario desea realizar una actualización de población o superficie al bucle `for` se le agrega la función `enumerate`, esta devuelve el índice y el valor de cada elemento al mismo tiempo, Al tener el índice, puedes acceder directamente a la posición exacta de la lista y modificar el diccionario con el método `.update()`, evitando errores al recorrer la estructura.

Proceso de búsqueda y validación de similitud

Cuando el usuario desea buscar un país o continente específico, el sistema no solo realiza una comparación directa, sino que pasa por un proceso de flexibilización y tolerancia a errores de tipeo.

En primer lugar, la entrada del usuario se estandariza eliminando espacios innecesarios y formateando el texto en formato de títulos. Posteriormente, el dato ingresado se envía a la función `buscar_parecido()`. Esta función actúa como un filtro inteligente que compara el intento del usuario contra las tuplas de referencia (`lista_paises` o `lista_continentes`); si detecta una palabra con un alto porcentaje de similitud (por ejemplo, "América" cuando el usuario escribió "america"), devuelve el término corregido y con la acentuación oficial de la base de datos.

Orden de diccionarios dentro de listas

Cuando el usuario le pide al programa que organice los países por nombre, población o superficie, se hace uso del método `sorted`, acompañado de la palabra clave `lambda` que permite crear funciones que se escriben en una sola línea de código y no tienen un nombre asignado, esto permite ordenar una lista de diccionarios o tuplas, en caso de agregarle el parámetro `reverse = True` la nueva lista se ordenara en forma inversa.

Estadísticas de población mínima y máxima

Para obtener los índices de población se utilizan las funciones `min` y `max` acompañadas de la palabra clave `lambda` ya que es necesario recorrer los diccionarios dentro de la lista y comparar el valor de una clave determinada dentro de ellos, sin esta función Python intentaría comparar los diccionarios completos entre sí, lo cual arrojaría un error (`TypeError`).

Manejo de Archivo.csv

Al iniciar el programa el sistema antes de solicitarle al usuario que ingrese una de las opciones del menú, intenta abrir el archivo llamado "países.csv" en modo lectura, este es observado por el bloque `Try-except`, en caso de que no exista el archivo el bloque captura el error (`FileNotFoundError`), genera un archivo con ese nombre y establece la primera línea del dataset para que sea posible comprender el contenido del archivo.

En caso de que sea la primera vez que se ejecuta el programa esto es vital, ya que genera un archivo que le permite al usuario guardar todos los datos que ingresa, de otra forma al cerrar el programa se perderían todos ingreso realizado, así mismo el programa cada vez que el

usuario agrega o actualiza la información, al finalizar la acción el programa actualiza los datos en el archivo, sobrescribiendo la información que contiene, de esta forma evita posibles errores y mantiene siempre actualizados los valores dentro del archivo.

Decisiones Técnicas y Arquitectura

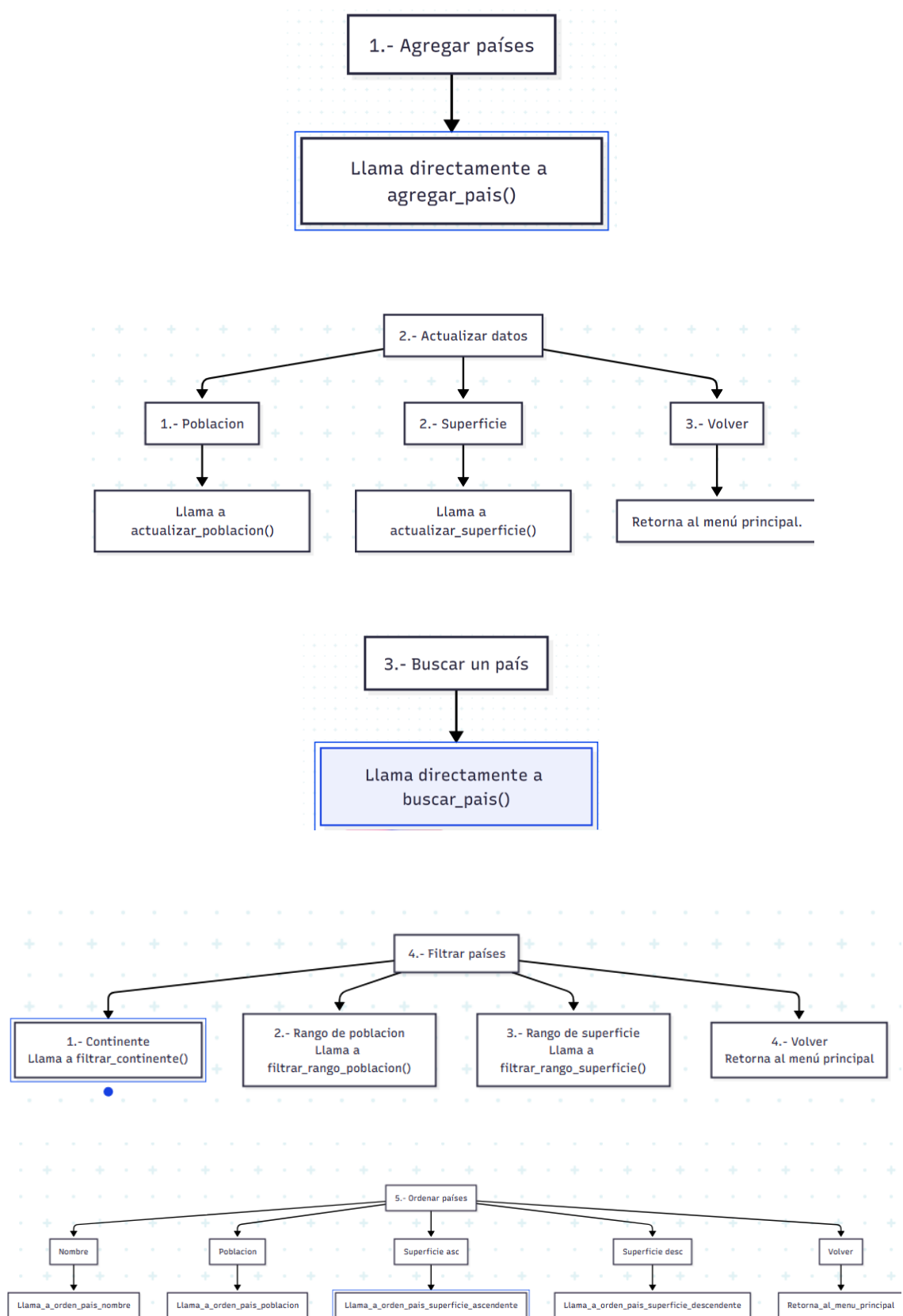
La arquitectura del sistema sigue un patrón de procedimientos modularizado, cada operación esta contenida en una función independiente, lo que permite reutilizar los menú y mantener el código organizado. La persistencia se logra mediante el archivo CSV que se actualiza en cada operación.

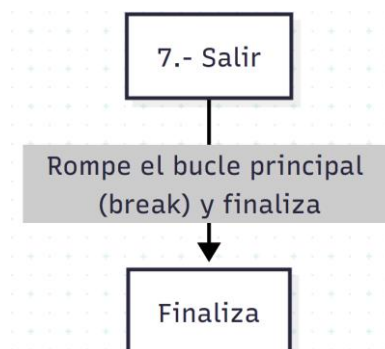
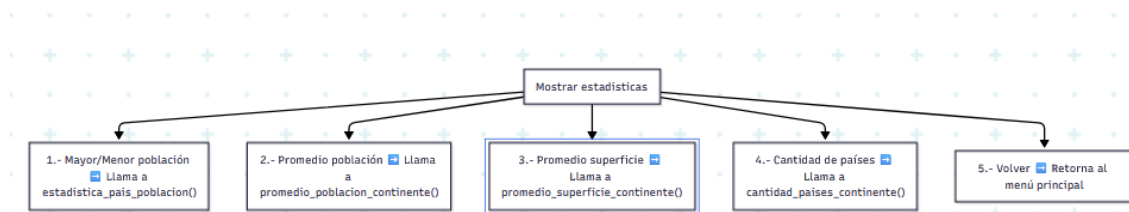
Esquema del menú

Este esquema te muestra exactamente qué funciones o submenús se disparan según la opción que elija el usuario en el Menú Principal:

- **1.- Agregar países** ➡ Llama directamente a `agregar_pais()`
- **2.- Actualizar datos**
 - 1.- Poblacion ➡ Llama a `actualizar_poblacion()`
 - 2.- Superficie ➡ Llama a `actualizar_superficie()`
 - 3.- Volver ➡ Retorna al menú principal.
- **3.- Buscar un país** ➡ Llama directamente a `buscar_pais()`
- **4.- Filtrar países**
 - 1.- Continente ➡ Llama a `filtrar_continente()`
 - 2.- Rango de poblacion ➡ Llama a `filtrar_rango_poblacion()`
 - 3.- Rango de superficie ➡ Llama a `filtrar_rango_superficie()`
 - 4.- Volver ➡ Retorna al menú principal.
- **5.- Ordenar países**
 - 1.- Nombre ➡ Llama a `orden_pais_nombre()`
 - 2.- Poblacion ➡ Llama a `orden_pais_poblacion()`
 - 3.- Superficie asc ➡ Llama a `orden_pais_superficie_ascendente()`
 - 4.- Superficie desc ➡ Llama a `orden_pais_superficie_descendente()`
 - 5.- Volver ➡ Retorna al menú principal.
- **6.- Mostrar estadísticas**
 - 1.- Mayor/Menor población ➡ Llama a `estadistica_pais_poblacion()`
 - 2.- Promedio población ➡ Llama a `promedio_poblacion_continente()`
 - 3.- Promedio superficie ➡ Llama a `promedio_superficie_continente()`
 - 4.- Cantidad de países ➡ Llama a `cantidad_paises_continente()`
 - 5.- Volver ➡ Retorna al menú principal.
- **7.- Salir** ➡ Rompe el bucle principal (`break`) y finaliza.

Organigrama de cada opción del menú





Gráficos realizado con la herramienta Mermaid.ai

Capturas de pantalla

A continuación se muestran ejemplos representativos de la ejecución del programa en consola, ilustrando las operaciones principales del sistema.

Ejemplo 1 – Ingreso de un país con validación

```

- Actualizacion del registro de paises -

a continuación se le solicitara que ingrese la cantidad de paises
que desea guardar en la base de datos, seguido de la informacion
en el orden establecido:
- Nombre.
- Poblacion.
- Superficie en Km².
- Continente.

Cuantos paises va a agregar a la lista?
ingrese la cantidad con numeros: 1
ingrese el nombre del pais: Monte Carlo
IMPORTANTE: estas agregando a la lista un pais que no existe,
asegurate de ingresarlo de forma correcta, revisa los acentos y espacios.

quieres agregarlo de todos modos? s/n: s
- ingrese los datos de MonteCarlo -
ingrese la poblacion actual:23445
ingrese la superficie en Km²: 12345
ingrese el continente en el que se encuentra: Europa

pais guardado con exito.
  
```

Ejemplo 2 – Ordenamiento por población (mayor a menor)

orden de países por población:

```
país: Andorra
continente: Europa
poblacion: 79824
-----

país: Antigua y Barbuda
continente: América
poblacion: 94298
-----

país: Barbados
continente: América
poblacion: 281635
-----

país: Bahamas
continente: América
poblacion: 409984
-----

país: Belice
continente: América
poblacion: 412198
```

Ejemplo 3 – Estadísticas de población.

```
-----
el país con menor poblacion es: Andorra
se encuentra en el continente: Europa
tiene una poblacion de: 79824 habitantes
en una superficie de: 468 Km²

el país con mayor poblacion es: India
se encuentra en el continente: Asia
tiene una poblacion de: 1417173173 habitantes
en una superficie de: 3287263 Km²
-----
```

Dificultades

- Durante la creación del programa fue importante estandarizar el ingreso de datos, nombre que llevarán las claves de diccionario, funciones y demás dado que el cambio en una letra o símbolo puede generar un problema a la hora de encontrar un dato en particular o llamar a una función, generando errores y dificultades.
- Una de las características fundamentales de la aplicación es la implementación de un sistema de lectura y escritura de archivos en formato CSV (países.csv). Esto dota al programa de persistencia, permitiendo que la información registrada no se pierda al finalizar la ejecución.
- Al iniciar el programa, se realiza una lectura automática que reconstruye la base de datos a partir del archivo físico, utilizando el manejo de excepciones (FileNotFoundError) para crear el archivo en caso de que no exista. Al cerrarse el programa, un bloque finally asegura la escritura y actualización de todos los datos en el archivo. Este proceso requirió la conversión de tipos de datos, transformando cadenas de texto del archivo CSV a números enteros y diccionarios dentro del entorno de Python.
- Bucles infinitos en menús secundarios: Al usar ciclos while True en los submenús, si el usuario ingresaba una opción no válida (como el 9), el programa se quedaba atrapado repitiendo el menú sin pedir un nuevo dato. Lo solucionamos agregando un input() directo dentro de cada ciclo para forzar al programa a detenerse y esperar.

- Validación de errores en datos ingresados: Nos aseguramos que el programa no fallara o se cerrara inesperadamente cuando el usuario se equivocaba; tuvimos que programar controles estrictos para frenar el ingreso de letras en campos numéricos (población/superficie) y de números en campos de texto (países/continentes).
- Errores lógicos (búsquedas y funciones): Tuvimos fallas lógicas difíciles de ver. Por ejemplo, la búsqueda de países con espacios (ej: "Costa Rica") fallaba porque el buscador eliminaba los espacios del usuario pero la base de datos los mantenía. Ambos casos se solucionaron usando variables intermedias y unificando los formatos de texto.
- Se implementó una función que `Buscar_parecido` para solucionar el tema de letras acentuadas, por si el usuario escribía America sin acento, lo tome como valido al continente.

Conclusiones

El desarrollo de este Trabajo Práctico Integrador nos permitió consolidar de manera práctica los conceptos teóricos fundamentales de la programación estructurada y la gestión de estructuras de datos dinámicas en Python. A través del diseño de este sistema de registro geográfico, logramos implementar un flujo de trabajo completo que abarca desde lectura y escritura de archivos csv, captura y validación de datos por parte del usuario hasta procesamientos de filtros, ordenamientos y cálculos estadísticos.

Asimismo, la integración y el uso de un **repositorio para el control de versiones** constituyó un pilar fundamental en nuestra metodología de trabajo. Esta herramienta no solo facilitó la colaboración sincronizada y eficiente entre los integrantes del equipo, sino que también nos permitió gestionar el historial de cambios, realizar un seguimiento estructurado del código frente a la resolución de errores y asegurar el despliegue óptimo de una versión final estable del software.

Referencias

- *Material de estudio de la catedra de Tecnicatura Universitaria en Programación. Universidad Tecnológica Nacional.*
- <https://www.worldometers.info/es/geografia/cuantos-paises-hay-en-el-mundo/>
- Python Software Foundation. (2024). *os — Miscellaneous operating system interfaces.* <https://docs.python.org/3/library/os.html>
- Python Software Foundation. (2024). *csv — CSV File Reading and Writing.* <https://docs.python.org/3/library/csv.html>
- Organización de las Naciones Unidas. (2024). *Estados Miembros.* <https://www.un.org/es/about-us/member-states>
- Organización de las Naciones Unidas. (2024). *Regiones geográficas.* <https://es.wikipedia.org/wiki/Continentes>
- *Página Web IA Gemini:* <https://gemini.google.com/>
- *Página Web IA Claude:* <https://claude.ai/new>